

3-实时消息推送和kafka集成

零声教育 C++Linux课程

零声教育 Darren

1. 快速开始

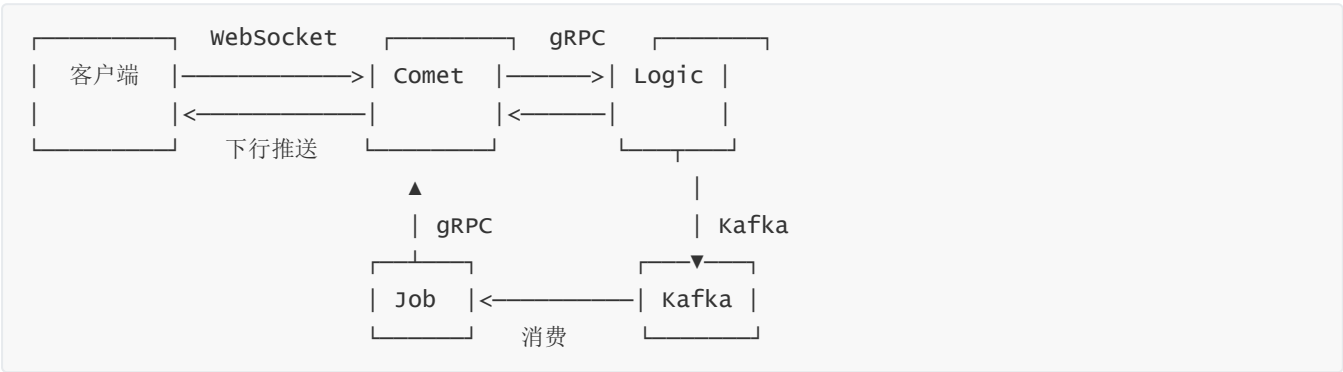
1.1 项目概述

Spark Push 03_msg_subset (消息子集) 在认证子集基础上，新增一对单聊消息功能。

核心功能

- ☒ WebSocket / HTTP 发送单聊消息
- ☒ Kafka 推送队列 (Logic → Kafka → Job → Comet)
- ☒ 会话管理 (Redis 轻量会话列表 + last_msg 快照)
- ☒ Kafka 消息队列
- ☒ Redis 在线路由 (基于连接维度，带 TTL)
- ☒ 未读计数 (Redis unread_count:* 递增维护)
- ☒ 消息 ACK 机制

服务架构

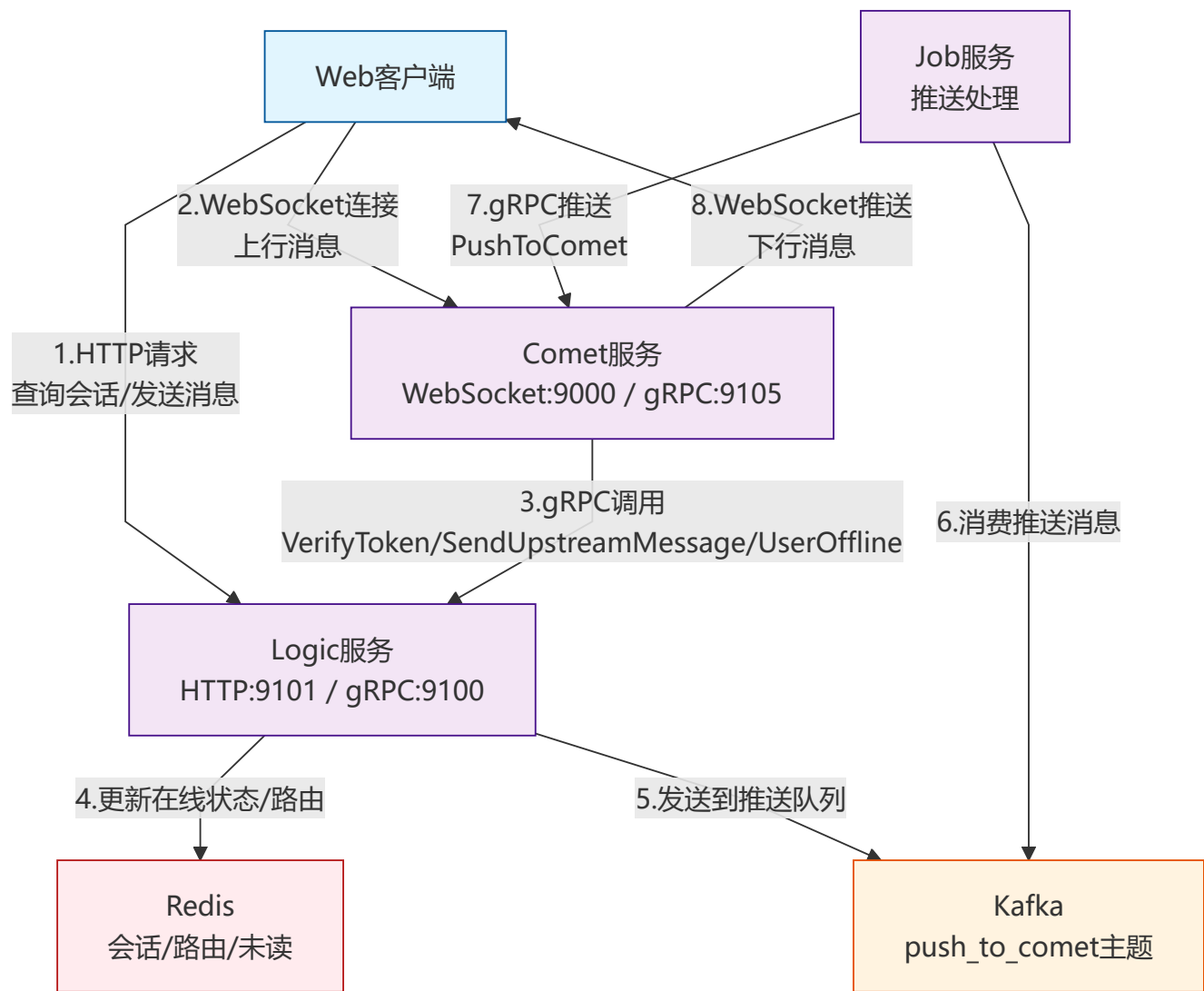


三个核心服务：

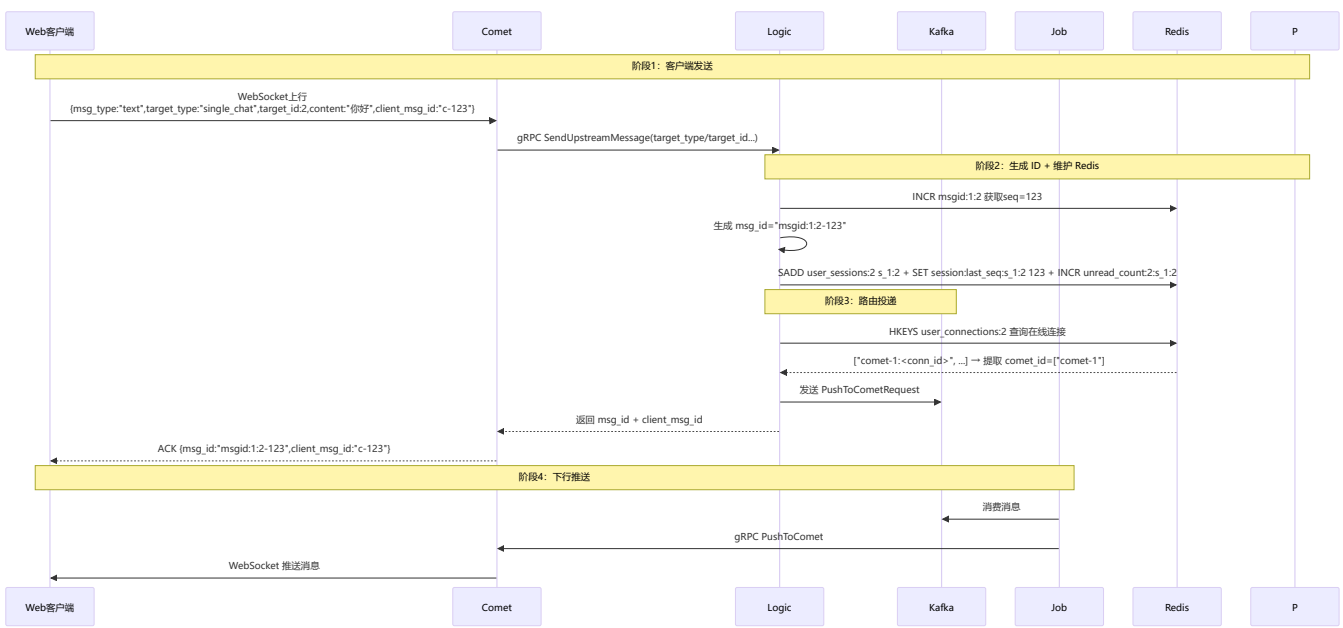
- **Logic**：业务逻辑（消息路由、持久化、会话管理）
- **Comet**：长连接网关（WebSocket、消息推送）
- **Job**：消息分发（Kafka 消费、推送到 Comet）

2. 核心设计

2.1 服务架构图



2.2 消息流转时序



3. 消息 ID 设计原理

3.1 为什么需要两种 ID?

在分布式即时通讯系统中，需要平衡**客户端体验**和**服务端一致性**：

需求场景	设计选择	理由
全局唯一标识	msgid （服务端生成）	Redis INCR 保证原子性、严格递增
客户端 ACK 匹配	client_msg_id （客户端生成）	客户端立即显示消息，收到 ACK 后更新状态
消息排序	msgid 中的 seq	服务端序号是排序的唯一依据
未读计数	Redis unread_count 递增维护	写入时 <code>INCRBY unread_count:{uid}:{session_id}</code> ，读取时 <code>O(1)</code>

3.2 msg_id：服务端唯一标识

格式设计

```
msgid:{user1}:{user2}-{seq}
例如：msgid:1:2-123
```

组成部分：

- `msgid`：前缀标识
- `user1`：较小的 user_id（保证 A→B 和 B→A 使用同一计数器）

- `user2`: 较大的 `user_id`
- `seq`: Redis INCR 原子递增的序号 (123)

生成流程

```
// 1. 确保 user1 < user2
int64_t user1 = std::min(from_user, to_user);
int64_t user2 = std::max(from_user, to_user);

// 2. Redis INCR 获取严格递增序号
int64_t seq = 0;
redis->INCR("msgid:" + std::to_string(user1) + ":" + std::to_string(user2), &seq);
// Redis: INCR msgid:1:2 → 返回 123

// 3. 拼接完整 msg_id
std::string msg_id = "msgid:" + std::to_string(user1) + ":" +
                    std::to_string(user2) + "-" + std::to_string(seq);
// 结果: msgid:1:2-123
```

Redis 命令演示

```
# 用户1给用户2发第一条消息
127.0.0.1:6379> INCR msgid:1:2
(integer) 1 # 第1条

# 用户1给用户2发第二条消息
127.0.0.1:6379> INCR msgid:1:2
(integer) 2 # 第2条

# 用户2给用户1发消息 (同一个key!)
127.0.0.1:6379> INCR msgid:1:2
(integer) 3 # 第3条

# 并发测试: 原子性保证
127.0.0.1:6379> INCR msgid:1:2
(integer) 4
127.0.0.1:6379> INCR msgid:1:2
(integer) 5
# 无重复, 严格递增 ✓
```

为什么不用 UUID?

方案	优点	缺点	是否采用
Redis INCR	• 严格递增 • 可读性强 • 支持未读计数	• 依赖 Redis	✓ 本项目采用

方案	优点	缺点	是否采用
UUID	- 全局唯一 - 无需中心化	- 无序性 - 存储开销大 - 无法未读计数	✗
雪花算法	- 趋势递增 - 高性能	- 需时钟同步 - 不适合未读计算	✗

3.3 client_msg_id：客户端匹配标识

格式设计

```
c-{timestamp}-{random}
例如：c-1701234567-abc123
```

使用场景

场景1：发送消息时立即显示

```
// 客户端代码
function sendMessage(content) {
  // 1. 生成 client_msg_id
  const clientMsgId = 'c-' + Date.now() + '-' + Math.random().toString(36);

  // 2. 立即在 UI 中显示"发送中"
  displayMessage({
    client_msg_id: clientMsgId,
    content: content,
    status: 'sending' // 发送中
  });

  // 3. 发送到服务端
  ws.send(JSON.stringify({
    msg_type: 'text',
    target_type: 'single_chat',
    target_id: 2,
    content: content,
    client_msg_id: clientMsgId
  }));
}

// 收到 ACK 时更新状态
ws.onmessage = function(event) {
  const data = JSON.parse(event.data);

  if (data.type === 'ack') {
    // 根据 client_msg_id 找到对应消息
    updateMessageStatus(data.client_msg_id, 'sent', data.msg_id);
    // UI 更新：发送中 ⌚ → 已发送 ✓
  }
}
```

```
};
```

场景2：防止重复发送

```
const pendingMessages = new Map();

function sendMessage(content) {
    const clientMsgId = 'c-' + Date.now() + '-' + Math.random().toString(36);

    // 检查是否已发送
    if (pendingMessages.has(clientMsgId)) {
        console.warn('Message already sent:', clientMsgId);
        return;
    }

    pendingMessages.set(clientMsgId, {content, timestamp: Date.now()});
    ws.send(JSON.stringify({...}));

    // 设置超时清理
    setTimeout(() => {
        pendingMessages.delete(clientMsgId);
    }, 60000);
}
```

3.4 msg_seq：从 msg_id 提取的序号

设计原则

⚠ 重要：msg_seq 不是客户端生成的，而是从服务端的 msg_id 中提取的序号

```
// 错误做法（客户端生成）❌
msg_seq = 1, 2, 3... // 客户端自己维护序号
// 问题：无法保证全局顺序，不同客户端可能重复

// 正确做法（从 msg_id 提取）✅
msg_id = "msgid:1:2-123"
msg_seq = 123 // 从 msg_id 提取的序号
// 优点：全局唯一、严格递增、可用于排序和未读计数
```

提取方法

```
// 从 msg_id 提取 seq
std::string msg_id = "msgid:1:2-123";
int64_t seq = 0;

size_t pos = msg_id.rfind('-'); // 找到最后一个 '-'
if (pos != std::string::npos) {
    seq = std::stoll(msg_id.substr(pos + 1)); // 提取 "123"
}

// 结果：seq = 123
```

用途

1. 消息排序:

```
SELECT * FROM message
WHERE session_id='s_1:2'
ORDER BY msg_seq  -- 按序号排序
LIMIT 20;
```

2. 未读计数:

```
// 本子集实现: 未读数直接读 Redis 递增计数
int64_t unread = 0;
redis_store->GetUnreadCount(user_id, session_id, &unread);
```

3. 增量拉取:

```
SELECT * FROM message
WHERE session_id='s_1:2' AND msg_seq > 100
ORDER BY msg_seq
LIMIT 20;
```

3.5 完整对比表

维度	msg_id	client_msg_id	msg_seq
生成方	服务端 (Logic)	客户端	服务端 (从 msg_id 提取)
格式	msgid:1:2-123	c-1701234567-abc	整数 (123)
唯一性	全局唯一	客户端内唯一	会话内唯一且递增
是否必需	✅ 必需	⚠️ 推荐 (ACK匹配)	✅ 必需 (从msg_id提取)
用途1	全局标识、防重	ACK 匹配	消息排序
用途2	日志追踪	客户端去重	未读计数
用途3	包含递增序号	状态管理	增量拉取

3.6 推荐使用模式

生产环境 (推荐)

```
{
  "msg_id": "msgid:1:2-123",      // 服务端生成 (必需)
  "msg_seq": 123,                // 从 msg_id 提取 (必需)
  "client_msg_id": "c-1701234567" // 客户端生成 (推荐)
}
```

- ✅ 功能完整：排序、未读、ACK、去重
- ✅ 体验良好：发送消息立即显示
- ⚠️ 复杂度适中

简化版（教学项目）

```
{  
  "msg_id": "msgid:1:2-123", // 服务端生成（必需）  
  "msg_seq": 123           // 从 msg_id 提取（必需）  
}
```

- ✅ 实现简单
- ✅ 核心功能可用
- ❌ 无 ACK 匹配（客户端体验稍差）

4. 单聊消息完整流程

4.1 WebSocket 上行流程

客户端发送

```
const ws = new WebSocket('ws://localhost:9000/ws?token=xxx');  
  
ws.onopen = function() {  
  ws.send(JSON.stringify({  
    // 必填: msg_type/target_type/target_id  
    msg_type: 'text',  
    target_type: 'single_chat',  
    target_id: 2,  
    content: '你好',  
    client_msg_id: 'c-' + Date.now()  
  }));  
};  
  
ws.onmessage = function(event) {  
  const data = JSON.parse(event.data);  
  
  if (data.type === 'ack') {  
    console.log('消息已确认: msg_id=', data.msg_id, 'client_msg_id=', data.client_msg_id);  
    // 根据 client_msg_id 更新 UI 中对应消息的状态  
  } else {  
    console.log('收到新消息: ', data);  
  }  
};
```

Comet 转发 (comet_server.cpp)

```
void CometServer::OnTextMessage(const TcpConnectionPtr& conn,
                                ConnContext& ctx,
                                const std::string& payload) {

    // 1. 解析上行消息
    UpstreamMessageMeta meta;
    if (!ParseUpstreamMessage(payload, &meta)) {
        SendError(conn, "invalid message format");
        return;
    }

    // 2. 构造 gRPC 请求
    UpstreamMessageRequest req;
    req.set_from_user_id(ctx.user_id);
    req.set_client_msg_id(meta.client_msg_id);
    req.set_content_json(payload);
    req.set_comet_id(comet_id_);
    req.set_conn_id(ctx.conn_id);
    req.set_received_time_ms(CurrentTimeMillis());
    req.set_msg_type(meta.msg_type);
    req.set_target_type(meta.target_type);
    req.set_target_id(meta.target_id);

    // 3. 调用 Logic
    UpstreamMessageReply rep;
    grpc::ClientContext rpc_ctx;
    auto status = logic_stub->SendUpstreamMessage(&rpc_ctx, req, &rep);

    if (!status.ok() || rep.error().code() != 0) {
        SendError(conn, "send failed");
        return;
    }

    // 4. 返回 ACK
    SendAck(conn, rep.message().msg_id(), rep.message().client_msg_id());
}
```

4.2 Logic 处理流程

SendUpstreamMessage (logic/grpc_service.cpp)

```
::grpc::Status LogicServiceImpl::SendUpstreamMessage(
    ::grpc::ServerContext*,
    const UpstreamMessageRequest* request,
    UpstreamMessageReply* response) {

    // 1. 参数验证
    int64_t from_user = request->from_user_id();
    int64_t to_user = request->target_id();
    if (from_user <= 0 || request->target_type() != "single_chat" || to_user <= 0) {
```

```

        return ERROR("invalid user_id");
    }

    // 2. 构造会话信息
    Session session;
    session.type = SessionType::kSingle;
    session.user1_id = std::min(from_user, to_user);
    session.user2_id = std::max(from_user, to_user);
    session.id = "s_" + std::to_string(session.user1_id) + ":" +
        std::to_string(session.user2_id);

    // 3. 生成消息 ID (Redis INCR)
    int64_t seq = 0;
    if (!redis_store->NextSingleMsgId(session.user1_id, session.user2_id, &seq)) {
        return ERROR("generate msg_id failed");
    }

    std::string msg_id = "msgid:" + std::to_string(session.user1_id) + ":" +
        std::to_string(session.user2_id) + "-" + std::to_string(seq);
    // 例如: msgid:1:2-123

    // 4. 补充消息内容
    int64_t now_ms = CurrentTimeMillis();
    std::string final_json = request->content_json();
    try {
        auto j = nlohmann::json::parse(final_json);
        j["msg_id"] = msg_id;
        j["msg_seq"] = seq; // 从 msg_id 提取的序号
        j["create_time"] = now_ms;
        j["from_user_id"] = from_user;
        j["client_msg_id"] = request->client_msg_id();

        // 查询发送方信息
        User u;
        if (user_dao->GetUserById(from_user, &u, nullptr)) {
            j["sender"] = {{"uid", from_user}, {"name", u.name}};
        }
        final_json = j.dump();
    } catch (...) {}

    // 5. 构造 ChatMessage
    ChatMessage cm;
    cm.set_msg_id(msg_id);
    cm.set_session_id(session.id);
    cm.set_msg_seq(seq);
    cm.set_sender_id(from_user);
    cm.set_timestamp_ms(now_ms);
    cm.set_msg_type("text");
    cm.set_content_json(final_json);
    cm.set_client_msg_id(request->client_msg_id());

    // 6. 更新 Redis 会话信息
    if (redis_store_) {

```

```

        redis_store_>AddUserSession(from_user, session.id);
        redis_store_>AddUserSession(to_user, session.id);
        redis_store_>SetSessionLastSeq(session.id, seq);
        redis_store_>IncrUnreadCount(to_user, session.id, 1);

        // 更新会话元数据（用于会话列表展示）
        std::string preview = content.size() > 50 ? content.substr(0, 50) : content;
        redis_store_>SetUserSessionMeta(from_user, session.id, msg_id, seq, "text", now_ms,
preview);
        redis_store_>SetUserSessionMeta(to_user, session.id, msg_id, seq, "text", now_ms,
preview);
    }

    // 7. 查询接收方路由
    std::vector<std::string> comets;
    if (redis_store_ && redis_store_>GetUserConnectionComets(to_user, &comets) &&
!comets.empty()) {
        // 在线推送
        for (const auto& comet_id : comets) {
            PushToCometRequest req;
            req.set_comet_id(comet_id);
            *req.mutable_message() = cm;
            req.add_targets()->set_user_id(to_user);

            std::string payload;
            req.SerializeToString(&payload);
            push_producer_>Send(comet_id, payload);
        }
    } else {
        LOG_INFO << "User " << to_user << " offline, skip push";
        // 离线占位（可选：发送空 comet_id 消息供下游处理）
        PushToCometRequest req;
        req.set_comet_id("");
        *req.mutable_message() = cm;
        std::string payload;
        if (req.SerializeToString(&payload)) {
            push_producer_>Send("", payload);
        }
    }
}

// 8. 填充响应
*response->mutable_message() = cm;
SetError(response->mutable_error(), 0, "ok");
return ::grpc::Status::OK;
}

```

4.3 Kafka 投递与消费

Logic 投递

```
// Topic: push_to_comet
// Key: comet_id (如 "comet-1")
// Value: PushToCometRequest (Protobuf)

bool KafkaProducer::Send(const std::string& key, const std::string& value) {
    return producer_->produce(
        topic_.get(),
        RdKafka::Topic::PARTITION_UA, // 自动分区
        RdKafka::Producer::RK_MSG_COPY,
        const_cast<char*>(value.data()),
        value.size(),
        &key,
        nullptr
    ) == RdKafka::ERR_NO_ERROR;
}
```

Job 消费并转发

```
void JobRunner::HandleMessage(const std::string& key, const std::string& value) {
    (void)key; // key 是 comet_id, 已包含在 value 的 PushToCometRequest 中

    // 提交到线程池异步处理 (避免阻塞 Kafka 消费线程)
    rpc_pool_.Submit([this, payload = value]() {
        PushToCometRequest req;
        if (!req.ParseFromString(payload)) {
            LOG_ERROR << "Failed to parse PushToCometRequest from Kafka";
            return;
        }
        ProcessPushRequest(req); // 调用内部方法处理
    });
}

void JobRunner::ProcessPushRequest(const PushToCometRequest& req) {
    // 根据 comet_id 获取对应的 gRPC Stub
    CometService::Stub* stub = GetStub(req.comet_id());
    if (!stub) {
        LOG_ERROR << "Unknown comet_id: " << req.comet_id();
        return;
    }

    PushToCometReply reply;
    grpc::ClientContext ctx;
    auto status = stub->PushToComet(&ctx, req, &reply);

    if (!status.ok()) {
        LOG_ERROR << "PushToComet RPC failed: " << status.error_message();
    } else if (reply.error().code() != 0) {

```

```

        LOG_ERROR << "PushToComet error: " << reply.error().message();
    }
}

```

4.4 Kafka Push/Pull 原理

4.4.1 Producer: Logic 推送消息 (Push)

核心配置:

```

// common/kafka_producer.cpp
bool KafkaProducer::Init(const std::string& brokers, const std::string& topic) {
    // 可靠性配置
    conf->set("acks", "all"); // 等待所有副本确认
    conf->set("enable.idempotence", "true"); // 幂等生产者（防重复）
    conf->set("retries", "5"); // 失败重试5次
    conf->set("retry.backoff.ms", "500"); // 重试间隔500ms

    // 性能优化
    conf->set("linger.ms", "5"); // 批量等待5ms
    conf->set("batch.num.messages", "1000"); // 批量大小1000条
    conf->set("compression.type", "lz4"); // 压缩算法

    // 创建 Producer 和 Topic
    producer_.reset(RdKafka::Producer::create(conf, errstr));
    topic_.reset(RdKafka::Topic::create(producer_.get(), topic_name_, ...));
}

```

发送消息:

```

bool KafkaProducer::Send(const std::string& key, const std::string& value) {
    // key = comet_id (用于分区)
    // value = PushToCometRequest 序列化后的 Protobuf 字节流
    RdKafka::ErrorCode err = producer_->produce(
        topic_.get(),
        RdKafka::Topic::PARTITION_UA, // 根据 key 自动选择分区
        RdKafka::Producer::RK_MSG_COPY,
        const_cast<char*>(value.data()),
        value.size(),
        &key, // 分区键: 相同 comet_id 的消息发到同一分区
        nullptr
    );
    producer_->poll(0); // 触发内部事件循环
    return err == RdKafka::ERR_NO_ERROR;
}

```

分区策略: 使用 `comet_id` 作为 key, Kafka 自动通过 hash 将相同 `comet_id` 的消息路由到同一分区, 保证消息顺序性。

4.4.2 Consumer: Job 拉取消息 (Pull)

核心配置:

```
// common/kafka_consumer.cpp
bool KafkaConsumer::Init(...) {
    conf->set("group.id", group_id);           // 消费者组 (负载均衡)
    conf->set("auto.offset.reset", "earliest"); // 从最早消息开始消费
    conf->set("enable.auto.commit", "false");  // 手动提交 offset
    conf->set("max.poll.interval.ms", "300000"); // 最大拉取间隔5分钟
    conf->set("session.timeout.ms", "45000");  // 心跳超时45秒

    consumer_.reset(RdKafka::KafkaConsumer::create(conf, errstr));
    consumer_->subscribe({topic_}); // 订阅 topic
}
```

消费循环:

```
void KafkaConsumer::Loop() {
    while (running_) {
        // 每次拉取等待最多1秒
        std::unique_ptr<RdKafka::Message> msg(consumer_->consume(1000));
        if (msg && msg->err() == RdKafka::ERR_NO_ERROR) {
            std::string key = msg->key() ? *msg->key() : "";
            std::string value(static_cast<const char*>(msg->payload()),
                              msg->len());

            // 回调处理消息
            callback_(key, value);

            // 手动提交 offset (确保处理成功后才提交)
            consumer_->commitAsync(msg.get());
        }
    }
}
```

Job 服务消费流程:

```
// job/service.cpp
void JobRunner::HandleMessage(const std::string& key, const std::string& value) {
    // 提交到线程池异步处理 (避免阻塞 kafka 消费线程)
    rpc_pool_.Submit([this, payload = value]() {
        PushToCometRequest req;
        req.ParseFromString(payload); // 反序列化 Protobuf
        ProcessPushRequest(req);      // 调用 Comet gRPC
    });
}
```

Pull 模型优势:

- ✔ 消费者控制消费速度 (背压控制)

-  批量拉取，减少网络开销
-  Offset 管理灵活（手动提交防止消息丢失）

4.5 gRPC 客户端/服务端设计

本项目中，Logic、Comet、Job 三个服务之间通过 gRPC 通信，各服务既是服务端也是客户端。

4.5.1 服务角色矩阵

服务	gRPC 服务端角色	gRPC 客户端角色
Logic	 提供 <code>VerifyToken</code> 、 <code>SendUpstreamMessage</code> 等	 无
Comet	 提供 <code>PushToComet</code> （接收下行消息）	 调用 Logic 的 <code>SendUpstreamMessage</code>
Job	 无	 调用 Comet 的 <code>PushToComet</code>

4.5.2 Logic gRPC 服务端

服务启动：

```
// logic/main.cpp
int RunLogic(const Config& cfg) {
    // 1. 创建服务实现类
    auto service = std::make_unique<LogicServiceImpl>(
        &user_dao, &producer, &redis_store, &conversation_store);

    // 2. 配置 gRPC 服务器
    grpc::ServerBuilder builder;
    std::string grpc_addr = "0.0.0.0:" + std::to_string(cfg.listen_port);
    builder.AddListeningPort(grpc_addr, grpc::InsecureServerCredentials());
    builder.RegisterService(service.get());

    // 3. 启动服务器（独立线程运行）
    std::unique_ptr<grpc::Server> server(builder.BuildAndStart());
    std::thread grpc_thread([&server]() { server->wait(); });

    // 4. HTTP 服务器在主线程运行
    muduo::net::EventLoop loop;
    HttpApiServer httpServer(&loop, ...);
    loop.loop(); // 阻塞
}
```

接口实现示例：

```
// logic/grpc_service.cpp
::grpc::Status LogicServiceImpl::SendUpstreamMessage(
    ::grpc::ServerContext*,
    const UpstreamMessageRequest* request,
    UpstreamMessageReply* response) {
```

```
// comet/comet_server.cpp
void CometServer::OnTextMessage(const TcpConnectionPtr& conn,
                                ConnContext& ctx,
```

```

                                const std::string& payload) {
// 1. 解析 websocket 上行消息
UpstreamMessageMeta meta;
ParseUpstreamMessage(payload, &meta);

// 2. 构造 gRPC 请求
UpstreamMessageRequest req;
req.set_from_user_id(ctx.user_id);
req.set_content_json(payload);
req.set_client_msg_id(meta.client_msg_id);
req.set_comet_id(comet_id_);
req.set_conn_id(ctx.conn_id);
req.set_received_time_ms(CurrentTimeMillis());
req.set_msg_type(meta.msg_type);
req.set_target_type(meta.target_type);
req.set_target_id(meta.target_id);

// 3. 调用 Logic 的 gRPC 接口
UpstreamMessageReply rep;
grpc::ClientContext rpc_ctx;
auto status = logic_stub->SendUpstreamMessage(&rpc_ctx, req, &rep);

// 4. 返回 ACK 给客户端
if (status.ok()) {
    SendAck(conn, rep.message().msg_id(), rep.message().client_msg_id());
}
}

// Comet 启动时创建 gRPC 客户端 Stub
CometServer::CometServer(EventLoop* loop, const Config& cfg) {
    auto channel = grpc::CreateChannel(cfg.logic_grpc_target,
                                       grpc::InsecureChannelCredentials());
    logic_stub_ = LogicService::NewStub(channel);
}

```

4.5.4 Job gRPC 客户端

动态 Stub 管理（支持多个 Comet 实例）：

```

// job/service.cpp
CometService::Stub* JobRunner::GetStub(const std::string& comet_id) {
    std::lock_guard<std::mutex> lock(stub_mu_);

    // 1. 查找缓存的 stub
    auto it = comet_stubs_.find(comet_id);
    if (it != comet_stubs_.end()) {
        return it->second.get();
    }

    // 2. 根据 comet_id 获取地址（从配置文件解析）
    auto addr_it = comet_addrs_.find(comet_id);
    if (addr_it == comet_addrs_.end()) {

```

```

        return nullptr; // 未配置该 comet_id
    }

    // 3. 创建新的 channel 和 stub
    auto channel = grpc::CreateChannel(addr_it->second,
                                      grpc::InsecureChannelCredentials());
    auto stub = CometService::NewStub(channel);
    auto* ptr = stub.get();
    comet_stubs_[comet_id] = std::move(stub);
    return ptr;
}

// 调用 Comet 推送消息
void JobRunner::ProcessPushRequest(const PushToCometRequest& req) {
    CometService::Stub* stub = GetStub(req.comet_id());
    if (!stub) {
        LOG_ERROR << "Unknown comet_id " << req.comet_id();
        return;
    }

    PushToCometReply reply;
    grpc::ClientContext ctx;
    auto status = stub->PushToComet(&ctx, req, &reply);

    if (!status.ok()) {
        LOG_ERROR << "PushToComet RPC failed: " << status.error_message();
    }
}

```

4.5.5 gRPC 调用链路总结

上行消息流:

客户端 WebSocket → Comet (WebSocket 服务器)



下行消息流:

Kafka → Job (Kafka Consumer)



4.6 Comet 连接管理机制

4.6.1 核心数据结构

Comet 通过 `user_conns_` 管理所有活跃的 WebSocket 连接：

```
// comet/comet_server.h
class CometServer {
private:
    // 核心数据结构：用户ID → 连接集合的映射
    std::unordered_map<int64_t, std::set<TcpConnectionPtr>> user_conns_;
    mutable std::mutex conns_mu_; // 保护并发访问
};
```

数据结构可视化：

user_conns_ (std::unordered_map)

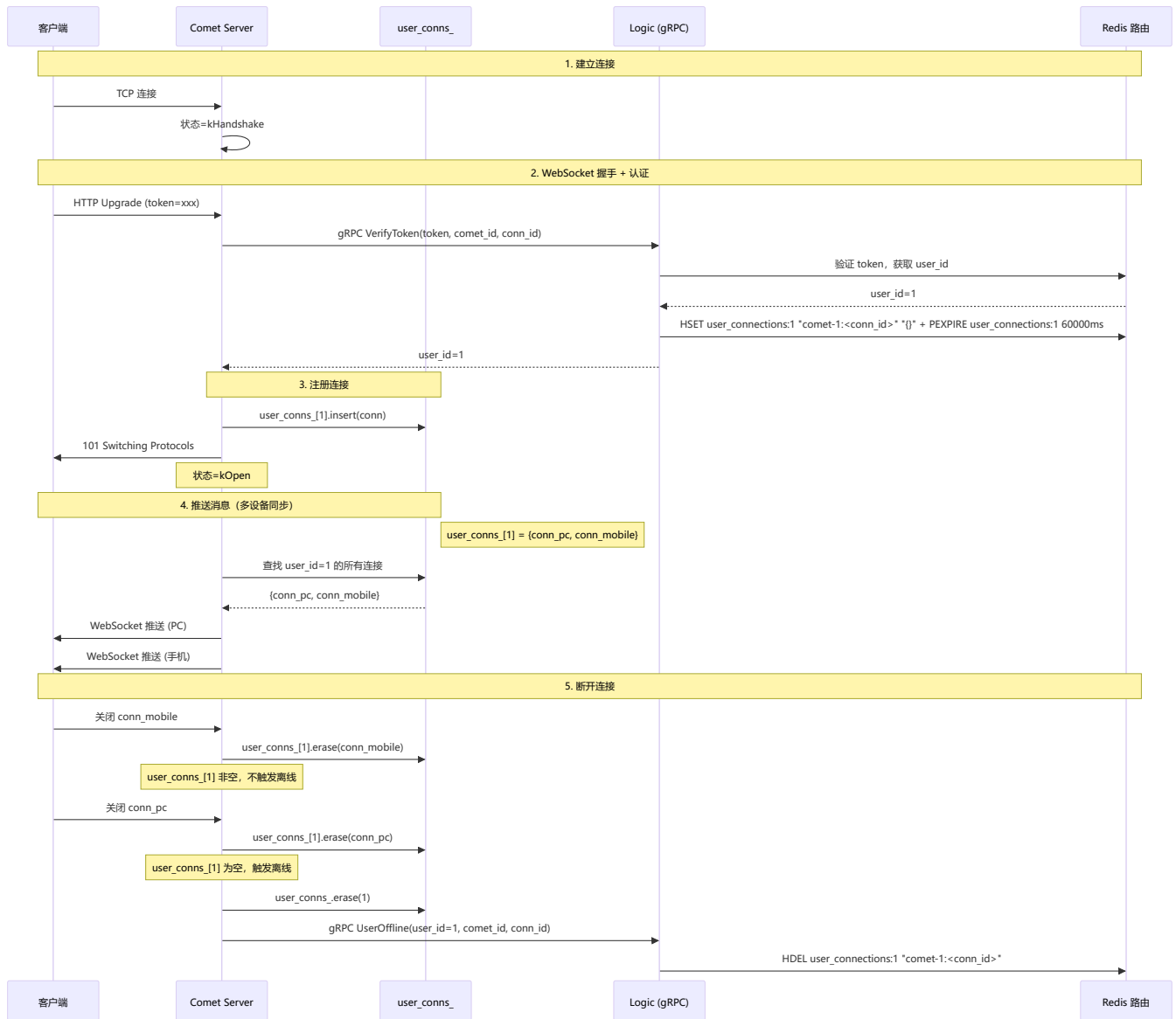
Key: user_id	Value: std::set<Connection>	
1 (用户A)	→ { conn_1, conn_2 }	← PC + 手机
2 (用户B)	→ { conn_3 }	← 单设备
3 (用户C)	→ { conn_4, conn_5, conn_6 }	← PC + 手机 + iPad

设计要点：

- 1. 使用 `std::set` 自动去重，防止同一连接重复添加
- 2. 支持一个用户同时有多个设备在线（多端登录）
- 3. 线程安全：所有操作都加锁保护

4.6.2 连接生命周期

流程图：



4.6.3 关键代码实现

添加连接（握手成功后）：

```

// comet/comet_server.cpp
void CometServer::HandleHandshake(const TcpConnectionPtr& conn, Buffer* buf) {
    // 1. 解析 token
    std::string token = ParseTokenFromHandshake(req);

    // 2. 调用 Logic 验证 token
    VerifyTokenRequest vreq;
    vreq.set_token(token);
    vreq.set_comet_id(comet_id_); // 告诉 Logic 本机 comet_id
    vreq.set_conn_id(conn_id);   // 连接唯一 ID (用于 Redis 在线连接登记与审计)
    VerifyTokenReply vrep;
    grpc::ClientContext ctx_rpc;
    auto status = logic_stub->VerifyToken(&ctx_rpc, vreq, &vrep);

    if (!status.ok() || vrep.error().code() != 0) {
        conn->shutdown(); // 认证失败, 关闭连接
    }
}

```

```

        return;
    }
    int64_t user_id = vrep.user_id();

    // 3. 完成 WebSocket 握手
    std::string accept_val = ComputeWebSocketAccept(ws_key);
    conn->send("HTTP/1.1 101 Switching Protocols\r\n...");

    // 4. 更新连接状态
    ConnContext ctx = std::any_cast<ConnContext>(conn->getContext());
    ctx.state = ConnContext::kOpen;
    ctx.user_id = user_id;
    conn->setContext(ctx);

    // 5. 将连接注册到 user_conns_ (关键步骤)
    {
        std::lock_guard<std::mutex> lock(conns_mu_);
        user_conns_[user_id].insert(conn); // 插入到该用户的连接集合
    }
    LOG_INFO << "WebSocket handshake done, user_id=" << user_id;
}

```

移除连接 (断开时) :

```

void CometServer::OnConnection(const TcpConnectionPtr& conn) {
    if (conn->connected()) {
        // 新连接建立 (初始化状态)
        ConnContext ctx;
        ctx.state = ConnContext::kHandshake;
        conn->setContext(ctx);
    } else {
        // 连接断开 (清理逻辑)
        int64_t offline_uid = 0;
        bool need_offline = false;

        try {
            auto ctx = std::any_cast<ConnContext>(conn->getContext());
            if (ctx.user_id > 0) {
                std::lock_guard<std::mutex> lock(conns_mu_);
                auto it = user_conns_.find(ctx.user_id);
                if (it != user_conns_.end()) {
                    // 从该用户的连接集合中移除
                    it->second.erase(conn);

                    // 如果该用户没有任何连接了, 标记为离线
                    if (it->second.empty()) {
                        user_conns_.erase(it);
                        offline_uid = ctx.user_id;
                        need_offline = true;
                    }
                }
            }
        } catch (const std::bad_any_cast&) {}
    }
}

```

```

        // 通知 Logic 用户离线（只有所有连接都断开时才通知）
        if (need_offline && offline_uid > 0) {
            NotifyUserOffline(offline_uid);
        }
    }
}

```

推送消息（多设备同时推送）：

```

void CometServer::PushToUsers(
    const ChatMessage& msg,
    const std::vector<int64_t>& user_ids) {

    // 1. 构造 websocket 文本帧
    std::string payload = msg.content_json();
    std::string frame = BuildWebSocketTextFrame(payload);

    // 2. 查找所有目标用户的所有连接
    std::vector<TcpConnectionPtr> conns;
    {
        std::lock_guard<std::mutex> lock(conns_mu_);
        for (int64_t uid : user_ids) {
            auto it = user_conns_.find(uid);
            if (it != user_conns_.end()) {
                // 将该用户的所有连接都加入推送列表
                for (const auto& c : it->second) {
                    conns.push_back(c);
                }
            }
        }
    }

    // 3. 批量发送到所有连接
    for (const auto& c : conns) {
        if (c->connected()) {
            c->send(frame); // 每个设备都会收到消息
        }
    }
}

```

4.6.4 多设备场景分析

场景 1：用户 A 在 PC 和手机同时在线

初始状态：

user_conns_[1] = {} （空）

步骤1：PC 连接

user_conns_[1] = { conn_pc }

Redis: HKEYS user_connections:1 = ["comet-1:<conn_id_pc>"]

步骤2: 手机连接

```
user_conns[1] = { conn_pc, conn_mobile }
```

Redis: HKEYS user_connections:1 = ["comet-1:<conn_id_pc>", "comet-1:<conn_id_mobile>"] (同一 comet 不同连接)

步骤3: 收到一条消息

```
PushToUsers(user_ids=[1])
```

→ 遍历 user_conns[1] = { conn_pc, conn_mobile }

→ conn_pc.send(frame)  PC 收到消息

→ conn_mobile.send(frame)  手机收到消息

步骤4: 手机断开

```
user_conns[1] = { conn_pc }
```

不触发 UserOffline (PC 还在线)

步骤5: PC 断开

```
user_conns[1] = {} → 删除 key
```

触发 UserOffline(user_id=1)

Redis: HDEL user_connections:1 "comet-1:<conn_id_pc>" (或 TTL 到期自动清理)

场景 2: 用户 A 在不同 Comet 上有多个设备

用户A在两台 Comet 上都有连接:

- comet-1: user_conns[1] = { conn_pc }

- comet-2: user_conns[1] = { conn_mobile }

Redis 路由:

HKEYS user_connections:1 = ["comet-1:<conn_id_pc>", "comet-2:<conn_id_mobile>"]

消息推送流程:

1. Logic 查询在线连接 → HKEYS user_connections:1 → 提取 comet_id 并去重得到 ["comet-1", "comet-2"]

2. 向 comet-1 发送 Kafka 消息 → PC 收到

3. 向 comet-2 发送 Kafka 消息 → 手机收到

场景 3: 防止重复推送

问题: 如果用户在同一 Comet 上有多个连接 (如多个浏览器标签页), 会收到多条消息吗?

答案: 是的, 每个连接都会收到。这是**设计预期**, 原因:

- 每个标签页/设备都是独立的显示界面
- 用户期望在所有打开的界面都看到新消息
- 客户端可根据 msg_id 去重 (本地缓存已显示的消息)

客户端去重示例:

```
const receivedMsgIds = new Set();

ws.onmessage = function(event) {
  const data = JSON.parse(event.data);
```

```
// 根据 msg_id 去重
if (receivedMsgIds.has(data.msg_id)) {
    console.log('重复消息，忽略');
    return;
}

receivedMsgIds.add(data.msg_id);
displayMessage(data); // 显示消息
};
```

4.6.5 设计优势与权衡

维度	设计方案	优势	劣势
数据结构	map<user, set<conn>>	支持多设备、自动去重、O(1) 查找	内存占用（需存储所有连接）
离线判断	所有连接断开时才通知 UserOffline	避免频繁通知、符合多设备语义	需要维护连接计数
推送策略	推送到用户所有连接	多端同步、用户体验好	同一消息多次传输
路由维度	以 comet_id 为维度存储路由	Job 可并行推送到多个 Comet	Redis 需维护路由映射
线程安全	所有操作加 mutex 保护	避免并发问题	锁竞争（高并发下瓶颈）

4.7 Comet 下行推送

CometServiceImpl::PushToComet

```
// gRPC 下行入口：根据 targets / session_id 将消息分发到本机连接集合。
::grpc::Status CometServiceImpl::PushToComet(
    ::grpc::ServerContext*,
    const ::sparkpush::PushToCometRequest* request,
    ::sparkpush::PushToCometReply* response) {
    // Comet 侧收到 job/logic 的下行推送请求：
    // 1. 如果 targets 列表非空，按 user_id 精确推送。
    // 2. 否则根据 message.session_id 约定进行房间/广播推送：
    //    - "r_<room_id>" 表示房间广播
    //    - "broadcast" 表示全服广播（仅限本 comet 内）
    // 3. PushToUsers/PushToRoom/PushToAll 均只在本机连接集合内分发，
    //    不跨节点路由，调用方需自行按 comet_id 分片。
    LOG_INFO << "PushToComet received for comet_id=" << request->comet_id()
        << " msg_id=" << request->message().msg_id();
    if (request->targets_size() > 0) {
        std::vector<int64_t> users;
        users.reserve(request->targets_size());
        for (const auto& t : request->targets()) {
```

```

        LOG_INFO << "PushToComet target user_id=" << t.user_id();
        users.push_back(t.user_id());
    }
    server_->PushToUsers(request->message(), users);
} else {
    // targets 为空: 按 session_id 约定处理
    const std::string& sid = request->message().session_id();
    if (sid == "broadcast") {
        server_->PushBroadcast(request->message());
    } else {
        LOG_WARN << "PushToComet targets empty but unsupported session_id=" << sid;
    }
}
response->mutable_error()->set_code(0);
response->mutable_error()->set_message("ok");
return ::grpc::Status::OK;
}

```

5. Redis 路由设计

5.1 核心问题

如何找到目标用户当前连接在哪个 Comet 服务器?

用户A(uid=1) 连接到 comet-1
 用户B(uid=2) 连接到 comet-2

当用户A给用户B发消息时:

Logic 需要知道用户B在 comet-2 上

→ 将消息投递到 comet-2

→ comet-2 推送给用户B

5.2 路由数据结构

Key: user_connections:{user_id}
 Type: HASH
 Field: {comet_id}:{conn_id}
 Value: "{}" (占位)
 TTL: PEXPIRE (例如 60s, 连接有心跳会刷新; 可配置: `comet\_redis\_ttl\_ms`)

示例:

HKEYS user_connections:1 = ["comet-1:comet-1-<ts>-1", "comet-1:comet-1-<ts>-2"]

HKEYS user_connections:2 = ["comet-1:...", "comet-2:..."] # 多设备/多节点

更完整示例: 同一用户在多个 comet, 且每个 comet 有多个连接

```
# 用户 2 同时在 comet-1 / comet-2 登录，并且每个 comet 都开了多个连接（多端/多标签页）
127.0.0.1:6379> HKEYS user_connections:2
1) "comet-1:comet-1-1765543000000-1"
2) "comet-1:comet-1-1765543000000-2"
3) "comet-2:comet-2-1765543005000-1"
4) "comet-2:comet-2-1765543005000-2"

# 应用侧提取 comet_id 并去重后得到：
# ["comet-1","comet-2"]
```

为什么用 HASH（按连接维度）？

- ☒ 支持多设备（同一用户可以在多个 Comet 上）
- ☒ 能区分同一 comet 上的不同连接（conn_id）
- ☒ 配合 TTL 可自动清理“异常断线残留路由”
- ☒ 原子操作（HSET/HDEL/HKEYS + PEXPIRE）

TTL 刷新机制（对应代码实现）

仅靠 TTL 还不够：需要在连接仍然活跃时持续刷新 TTL，否则即使在线也可能因过期被误清理。

本项目中，Comet 会在连接活跃时对 `user_connections:{uid}` 执行 `PEXPIRE` 刷新（典型触发：ping/pong 或业务消息）。

代码位置：`comet/comet_server.cpp` (`CometServer::RefreshUserTtl`)

```
void CometServer::RefreshUserTtl(int64_t user_id, const std::string& conn_id) {
    if (!redis_ready_ || user_id <= 0 || conn_id.empty()) return;
    auto guard = redis_pool_.Acquire(0);
    redisContext* ctx = guard.get();
    if (!ctx) return;
    std::string key = "user_connections:" + std::to_string(user_id);
    redisReply* reply = (redisReply*)redisCommand(ctx, "PEXPIRE %s %ld", key.c_str(),
static_cast<long long>(redis_ttl_ms_));

    if (reply) freeReplyObject(reply);
}
```

5.3 路由操作

添加路由（用户上线）

```
// key: user_connections:{uid}
// field: "{comet_id}:{conn_id}"
bool RedisStore::UpsertUserConnection(int64_t user_id,
    const std::string& comet_id,
    const std::string& conn_id,
    int64_t ttl_ms) {
    HSET user_connections:{uid} "{comet_id}:{conn_id}" "{}";
    PEXPIRE user_connections:{uid} ttl_ms;
```

```

}

# 用户1连接到 comet-1 (conn_id=abc)
127.0.0.1:6379> HSET user_connections:1 "comet-1:abc" "{}"
(integer) 1
127.0.0.1:6379> PEXPIRE user_connections:1 60000
(integer) 1

```

查询路由（发送消息时）

```

// 先 HKEYS, 得到 field 列表: ["comet-1:abc","comet-2:def"...]
// 再按 ":" 分割并去重得到 comet_id 列表: ["comet-1","comet-2"...]
bool RedisStore::GetUserConnectionComets(int64_t user_id,
                                          std::vector<std::string>* comets) {
    HKEYS user_connections:{uid};
}

# 查询用户2在线在哪些 comet 上（返回的是 field, 需要在应用侧提取 comet_id 并去重）
127.0.0.1:6379> HKEYS user_connections:2
1) "comet-1:abc"
2) "comet-2:def"

```

删除路由（用户下线）

```

bool RedisStore::RemoveUserConnection(int64_t user_id,
                                       const std::string& comet_id,
                                       const std::string& conn_id) {
    HDEL user_connections:{uid} "{comet_id}:{conn_id}";
}

```

触发时机：

- **正常断开**：WebSocket 连接断开时，Comet 调用 Logic 的 `UserOffline(user_id, comet_id, conn_id)` 精确清理
- **异常断开**：若没有触发离线回调，Redis 的 `PEXPIRE` 会在 TTL 到期后自动清理整张 `user_connections:{uid}`（心跳会刷新 TTL）

```

# 用户1断开 comet-1 的某个连接 (conn_id=abc)
127.0.0.1:6379> HDEL user_connections:1 "comet-1:abc"
(integer) 1

```

6. 数据库设计

说明：本子集（03_msg_subset）**未直接接入 MySQL 落库**

以下 MySQL 表结构属于扩展参考（用于你后续从 kafka 消费落库时对齐字段/索引）。

6.1 message 表（消息内容）

```
CREATE TABLE `message` (  
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,  
  `msg_id` VARCHAR(128) NOT NULL COMMENT '消息ID (msgid:1:2-123)',  
  `session_id` VARCHAR(64) NOT NULL COMMENT '会话ID',  
  `msg_seq` BIGINT NOT NULL DEFAULT 0 COMMENT '消息序号 (从msg_id提取)',  
  `sender_id` BIGINT NOT NULL COMMENT '发送方用户ID',  
  `msg_type` VARCHAR(32) NOT NULL DEFAULT 'text',  
  `content_json` TEXT NOT NULL COMMENT '消息内容JSON',  
  `timestamp_ms` BIGINT NOT NULL COMMENT '时间戳 (毫秒)',  
  `client_msg_id` VARCHAR(128) DEFAULT NULL COMMENT '客户端消息ID',  
  `created_at` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `uk_msg_id` (`msg_id`), -- 防重复  
  KEY `idx_session_seq` (`session_id`, `msg_seq`), -- 按序号查询  
  KEY `idx_session_time` (`session_id`, `timestamp_ms`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

关键索引说明：

- `uk_msg_id`：防止重复插入（幂等性）
- `idx_session_seq`：支持按序号分页查询（`WHERE session_id=? AND msg_seq > ? ORDER BY msg_seq LIMIT 20`）

6.2 Redis 会话管理（本节课实现）

⚠ **重要说明：**本项目的会话管理完全基于 Redis，不使用 MySQL 存储会话信息。

会话列表存储

```
# 用户的会话列表（SET 类型，自动去重）  
Key:   user_sessions:{user_id}  
Type:  SET  
Value: {session_id1, session_id2, ...}
```

示例：

```
user_sessions:1 = {"s_1:2", "s_1:3"} # 用户1的会话列表
```

操作示例：

```
// 添加会话到用户列表
bool RedisStore::AddUserSession(int64_t user_id, const std::string& session_id) {
    std::string key = "user_sessions:" + std::to_string(user_id);
    redisReply* reply = (redisReply*)redisCommand(ctx, "SADD %s %s", key.c_str(),
session_id.c_str());
    // ...
}

// 查询用户的所有会话
bool RedisStore::ListUserSessions(int64_t user_id, std::vector<std::string>* sessions) {
    std::string key = "user_sessions:" + std::to_string(user_id);
    redisReply* reply = (redisReply*)redisCommand(ctx, "SMEMBERS %s", key.c_str());
    // ...
}
```

会话最新消息序号

```
# 会话的最新消息序号
Key:    session:last_seq:{session_id}
Type:   STRING (整数)
Value:  最新消息的 msg_seq

示例:
session:last_seq:s_1:2 = "123" # 会话 s_1:2 的最新消息序号为 123
```

操作示例:

```
// 更新会话最新消息序号
bool RedisStore::SetSessionLastSeq(const std::string& session_id, int64_t last_seq) {
    redisReply* reply = (redisReply*)redisCommand(
        ctx, "SET session:last_seq:%s %lld", session_id.c_str(), last_seq);
    // ...
}

// 查询会话最新消息序号
bool RedisStore::GetSessionLastSeq(const std::string& session_id, int64_t* last_seq) {
    redisReply* reply = (redisReply*)redisCommand(
        ctx, "GET session:last_seq:%s", session_id.c_str());
    // ...
}
```

会话元数据（可选）

会话元数据（用于会话列表展示）

Key: user_session_meta:{user_id}:{session_id}

Type: HASH

Value: {last_msg_id, last_msg_seq, last_msg_type, last_time_ms, last_preview}

示例:

```
user_session_meta:1:s_1:2 = {
  "last_msg_id": "msgid:1:2-123",
  "last_msg_seq": "123",
  "last_msg_type": "text",
  "last_time_ms": "1701234567890",
  "last_preview": "你好"
}
```

操作示例:

```
// 更新会话元数据
bool RedisStore::SetUserSessionMeta(int64_t user_id, const std::string& session_id,
                                     const std::string& last_msg_id, int64_t last_msg_seq,
                                     const std::string& last_msg_type, int64_t last_time_ms,
                                     const std::string& last_preview) {
    std::string key = "user_session_meta:" + std::to_string(user_id) + ":" + session_id;
    redisReply* reply = (redisReply*)redisCommand(
        ctx, "HMSET %s last_msg_id %s last_msg_seq %lld last_msg_type %s last_time_ms %lld
last_preview %s",
        key.c_str(), last_msg_id.c_str(), last_msg_seq,
        last_msg_type.c_str(), last_time_ms, last_preview.c_str());
    // ...
}
```

6.3 Redis 未读计数管理

本项目使用 Redis 递增计数器实现未读数管理（O(1) 读写性能）：

未读计数存储

用户在某会话的未读消息数

Key: unread_count:{user_id}:{session_id}

Type: STRING（整数）

Value: 未读消息数

示例:

```
unread_count:2:s_1:2 = "5" # 用户2在会话 s_1:2 有5条未读消息
```

操作示例:

```
// 增加未读数（收到新消息时）
```

```

bool RedisStore::IncrUnreadCount(int64_t user_id, const std::string& session_id, int64_t
delta) {
    std::string key = "unread_count:" + std::to_string(user_id) + ":" + session_id;
    redisReply* reply = (redisReply*)redisCommand(ctx, "INCRBY %s %lld", key.c_str(),
delta);
    // ...
}

// 查询未读数
bool RedisStore::GetUnreadCount(int64_t user_id, const std::string& session_id, int64_t*
value) {
    std::string key = "unread_count:" + std::to_string(user_id) + ":" + session_id;
    redisReply* reply = (redisReply*)redisCommand(ctx, "GET %s", key.c_str());
    // ...
}

// 清空未读数（用户已读时）
bool RedisStore::ClearUnreadCount(int64_t user_id, const std::string& session_id) {
    std::string key = "unread_count:" + std::to_string(user_id) + ":" + session_id;
    redisReply* reply = (redisReply*)redisCommand(ctx, "DEL %s", key.c_str());
    // ...
}

```

Redis 命令演示

```

# 用户2收到新消息，未读数 +1
127.0.0.1:6379> INCRBY unread_count:2:s_1:2 1
(integer) 1

# 再收到一条
127.0.0.1:6379> INCRBY unread_count:2:s_1:2 1
(integer) 2

# 查询未读数
127.0.0.1:6379> GET unread_count:2:s_1:2
"2"

# 用户已读，清空未读数
127.0.0.1:6379> DEL unread_count:2:s_1:2
(integer) 1

127.0.0.1:6379> GET unread_count:2:s_1:2
(nil) # 返回 nil，客户端解释为 0

```

扩展：已读位置模型（可选实现）

如果需要更精确的未读管理，可以引入"已读位置"模型：

```
# 用户在会话中已读到的最大序号
Key:   read:{session_id}:{user_id}
Type:  STRING (整数)
Value: 已读序号
```

示例:

```
read:s_1:2:2 = "118" # 用户2在会话 s_1:2 已读到序号 118
```

```
# 未读数计算公式:
```

```
unread = session:last_seq - read_seq
```

```
# 例如: 123 - 118 = 5 条未读
```

⚠ 注意: 本项目默认使用递增计数器方案，未实现已读位置模型。

7. 构建与运行

7.1 编译

```
cd /home/alientek/0voice/spark_push/03_msg_subset
mkdir build && cd build
cmake ..
make -j$(nproc)
```

7.2 启动 Redis

```
# 启动 Redis (本项目核心依赖)
redis-server --port 6379 --daemonize yes

# 验证连接
redis-cli ping

# 输出: PONG
```

7.3 启动 Kafka

```
# 启动 Zookeeper
cd /path/to/kafka_2.13-3.7.0
# 前台运行(先确定前台运行正常后再改为后台运行)
bin/kafka-server-start.sh config/kraft/server-local.properties

# 如果需要后台运行则(适合测试环境后台部署):
nohup bin/kafka-server-start.sh config/kraft/server-local.properties > kafka.log 2>&1 &
# 然后使用命令参考日志
tail -f kafka.log
```

```
# 如果报错，重新初始化kafka的缓存目录（删除/tmp/kraft-combined-logs，这种方式这是项目测试使用，别在生产环境这么删除）
# 1. 生成 cluster ID（只需一次）
CLUSTER_ID=$(bin/kafka-storage.sh random-uuid)

# 2. 格式化 log.dirs 目录（会创建 meta.properties）
bin/kafka-storage.sh format -t $CLUSTER_ID -c config/kraft/server-local.properties
```

7.4 启动服务

```
# 终端1: Comet 服务
./comet/comet_server ../conf/comet.conf

# 终端2: Logic 服务
./logic/logic_server ../conf/logic.conf

# 终端3: Job 服务
./job/job_server ../conf/job.conf
```

8. 功能测试

8.1 注册用户

```
# 注册用户1
curl -X POST http://localhost:9101/api/register \
  -H "Content-Type: application/json" \
  -d '{"account":"user1","password":"123456","name":"用户A"}'

# 注册用户2
curl -X POST http://localhost:9101/api/register \
  -H "Content-Type: application/json" \
  -d '{"account":"user2","password":"123456","name":"用户B"}'
```

tk-1-1765459036

tk-2-1765459059

8.2 登录获取 Token

```
curl -X POST http://localhost:9101/api/login
```

说明：当前版本 `POST /api/login` 返回 `501 login not implemented`，测试请直接使用 **注册接口返回的 token**（`/api/register` 会下发 token 并写入 Redis `token:<token>`）。

8.3 WebSocket 连接

```
# 用户1连接
websocat "ws://localhost:9000/ws?token=tk-1-xxxxx"
websocat "ws://localhost:9000/ws?token=tk-1-1765542927"

# 用户2连接（另一终端）
websocat "ws://localhost:9000/ws?token=tk-2-yyyyy"
websocat "ws://localhost:9000/ws?token=tk-2-1765542931"
```

8.4 发送消息

在用户1的终端输入：

```
{"msg_type":"text","target_type":"single_chat","target_id":2,"content":"你好","client_msg_id":"c1-123"}
```

在用户2的终端输入：

```
{"msg_type":"text","target_type":"single_chat","target_id":1,"content":"今晚要上课","client_msg_id":"c2-133"}
```

预期结果：

1. 用户1收到 ACK：

```
{"type":"ack","msg_id":"msgid:1:2-1","client_msg_id":"c1-123"}
```

2. 用户2收到消息：

```
{
  "client_msg_id": "c1-123",
  "content": "你好",
  "create_time": 1765542967276,
  "from_user_id": 1,
  "msg_id": "msgid:1:2-1",
  "msg_seq": 1,
  "msg_type": "text",
  "target_id": 2,
  "target_type": "single_chat"
}
```

字段	说明
client_msg_id	客户端生成的唯一消息 ID，用于去重或确认
content	消息正文（此处为文本）

字段	说明
<code>create_time</code>	消息创建时间戳（毫秒），对应北京时间： 2025-12-12 20:36:07.276
<code>from_user_id</code>	发送者用户 ID
<code>msg_id</code>	服务端分配的全局唯一消息 ID
<code>msg_seq</code>	消息序列号（用于排序、同步）
<code>msg_type</code>	消息类型（如 <code>text</code> , <code>image</code> , <code>file</code> 等）
<code>target_id</code>	接收方 ID（用户 ID 或群组 ID/房间ID）
<code>target_type</code>	目标类型（ <code>single_chat</code> 单聊 / <code>group_chat</code> 群聊或者 <code>room_chat</code> 房间）

8.5 HTTP 发送消息

```
curl -X POST http://localhost:9101/api/message/send \
-H "Authorization: Bearer tk-1-1765542927" \
-H "Content-Type: application/json" \
-d '{
  "msg_type": "text",
  "target_type": "single_chat",
  "target_id": 2,
  "content": "HTTP消息（来自 HTTP）",
  "client_msg_id": "c1-456"
}'
```

8.6 查询会话列表

用户1查询sessions

```
curl -X POST http://localhost:9101/api/session/list_single \
-H "Content-Type: application/json" \
-H "Authorization: Bearer tk-1-1765542927" \
-d '{}'
```

响应:

```
{
  "code": 0,
  "message": "ok",
  "data": {
    "sessions": [
      {
        "session_id": "s_1:2",
        "peer_user_id": 2,
        "last_msg_seq": 65,

```

```

        "last_msg_id": "msgid:1:2-65",
        "last_msg_type": "text",
        "last_time_ms": 1765543243493,
        "last_preview": "HTTP消息（来自 HTTP）"
    }
]
}
}

```

用户1查询sessions

```

curl -X POST http://localhost:9101/api/session/list_single \
-H "Content-Type: application/json" \
-H "Authorization: Bearer tk-2-1765542931" \
-d '{}'

```

响应:

```

{
  "code": 0,
  "message": "ok",
  "data": {
    "sessions": [
      {
        "session_id": "s_1:2",
        "peer_user_id": 1,
        "last_msg_seq": 65,
        "last_msg_id": "msgid:1:2-65",
        "last_msg_type": "text",
        "last_time_ms": 1765543243493,
        "last_preview": "HTTP消息（来自 HTTP）"
      }
    ]
  }
}

```

8.7 查询未读数

```

curl -X POST http://localhost:9101/api/session/unread \
-H "Content-Type: application/json" \
-H "Authorization: Bearer tk-1-1765542927" \
-d '{"session_id":"s_1:2"}'

```

说明：在线 ≠ 已读

- **未读数维护方式**：本子集使用 Redis 计数器 `unread_count:{user_id}:{session_id}` 递增维护；发送消息时对接收方执行 `INCRBY`。
- **不会自动清零**：用户 WebSocket 在线/已登录并不会自动将未读清零；需要“标记已读”动作才会清零（或记录 `read_seq`）。

8.7.1 标记已读（清零未读数）

新增接口： `POST /api/session/mark_read`

需要携带 `Authorization: Bearer <token>`，用于从 token 解析 `user_id`。

方式1：只清零未读

```
curl -X POST http://localhost:9101/api/session/mark_read \  
-H "Content-Type: application/json" \  
-H "Authorization: Bearer tk-1-1765542927" \  
-d '{"session_id":"s_1:2"}'
```

方式2：清零未读 + 记录 `read_seq`（可选）

```
curl -X POST http://localhost:9101/api/session/mark_read \  
-H "Content-Type: application/json" \  
-H "Authorization: Bearer tk-1-1765542927" \  
-d '{"session_id":"s_1:2","read_seq":65}'
```

验证：清零后再次查询

```
curl -X POST http://localhost:9101/api/session/unread \  
-H "Content-Type: application/json" \  
-H "Authorization: Bearer tk-1-1765542927" \  
-d '{"session_id":"s_1:2"}'
```

9. 常见问题

Q1: 为什么 `msg_seq` 要从 `msg_id` 提取？

A: 因为消息排序必须依赖服务端的严格递增序号，客户端传入的序号无法保证全局顺序。

Q2: `client_msg_id` 是必需的吗？

A: 不是必需的，但强烈推荐。没有 `client_msg_id`，客户端无法精确匹配 ACK，用户体验会下降。

Q3: 如果不使用 Redis，如何生成 `msg_id`？

A: 可以使用数据库自增 ID 或雪花算法，但需要注意：

- 数据库自增：性能瓶颈

- 雪花算法：实现复杂，且需要保证序号严格递增

Q4: Kafka 消息丢失怎么办？

A:

1. 配置 `acks=all`（生产者等待所有副本确认）
2. 启用持久化（`log.retention.hours`）
3. Job 服务可以重启后继续消费（Kafka 保留消费位置）

Q5: Comet 崩溃后路由残留怎么办？

A: 实现路由 TTL + 心跳机制：

- Redis 使用 `PEXPIRE user_connections:{uid} <ttl_ms>`（默认 60s，可配置：`comet_redis_ttl_ms`）
- Comet 收到 ping/pong 或业务消息时刷新 TTL（保活）
- Comet 侧有空闲连接回收：**时间戳 + 分片惰性扫描**
 - 连接活跃时仅更新 `last_active_ms`
 - `checkIdleConnections()` 由 `runEvery` 定时触发，但**每次只扫描一个分片**（按 `user_id % 分片数`），避免全量 $O(N)$ 扫描抖动
 - 判定条件：`now_ms - last_active_ms > idle_timeout_ms`（可配置：`comet_idle_timeout_ms`）
 - 回收延迟上界约为：`tick_interval * 分片数`（例如 $1s * 10 = 10s$ ）
- Comet 崩溃/异常断线后，TTL 到期会自动清理残留的 `user_connections:{uid}`

空闲连接回收：时间戳 + 分片惰性扫描

1. 设计背景

为什么需要空闲连接回收？

即使客户端正常，某些场景下连接可能“僵死”（网络异常、客户端挂起、防火墙静默丢包等），导致：

- Comet 持续占用内存/文件描述符
- Redis 路由残留，后续消息无法送达
- 客户端误以为连接正常，实际已失效

解决方案需求：

1. 定期检测“长时间无活动”的连接并关闭
2. 避免对高频活跃连接造成性能抖动
3. 可配置超时阈值和检测频率

2. 方案对比

方案	说明	优点	缺点	复杂度
方案1：全量扫描	每 N 秒扫描所有连接，判断 <code>now - last_active > timeout</code>	实现简单	连接数大时每次扫描 O(N)，CPU 抖动明显；锁竞争严重	O(N)
方案2：per-connection 定时器	每个连接有活动时重置独立定时器	精确到期触发，无周期扫描	高频活跃连接频繁 cancel/runAfter；定时器队列可能膨胀	O(log N) per 活动
方案3：时间戳 + 分片惰性扫描（当前采用）	仅更新时间戳；定时扫描但 每次只检查 1 个分片	单次复杂度 O(N/shards)；活跃时零定时器开销	发现延迟有上界 (tick * shards)	O(N/shards)

为什么选方案3？

在 IM 场景中：

- **活跃连接占比高**：大部分连接会有定期 ping/pong，方案2 的 cancel/runAfter 开销不划算
- **允许一定延迟**：空闲回收延迟 10~20 秒可接受（相比超时阈值通常 60s）
- **锁粒度可控**：分片后单次持锁时间短，减少对 `PushToUsers` 等热路径的影响

3. 实现细节

3.1 时间戳更新

每当连接"活跃"时，仅更新 `ConnContext::last_active_ms`（无需操作定时器）：

```
// 握手成功
ctx.last_active_ms = now_ms;

// 收到 ping
ctx.last_active_ms = now_ms;

// 收到 pong
ctx.last_active_ms = now_ms;

// 收到业务消息
ctx.last_active_ms = now_ms;
```

3.2 分片算法

将所有用户连接按 `user_id` 分散到 `idle_scan_shards_` 个虚拟分片（默认 10）：

```
int shard_id = user_id % idle_scan_shards_;
```

每次扫描只处理**当前分片索引**对应的用户：

```

// CometServer 成员变量
int idle_scan_shards_{10};           // 分片总数
int idle_scan_shard_idx_{0};         // 当前扫描的分片索引

void CometServer::CheckIdleConnections() {
    int64_t now_ms = /* 当前时间戳 */;
    std::vector<TcpConnectionPtr> to_close;

    {
        std::lock_guard<std::mutex> lock(conns_mu_);

        // 只扫描"当前分片"的用户
        for (auto& kv : user_conns_) {
            int64_t user_id = kv.first;

            // 分片过滤: 跳过不属于当前分片的用户
            if (user_id % idle_scan_shards_ != idle_scan_shard_idx_) {
                continue;
            }

            // 检查该用户的所有连接
            for (auto& conn : kv.second) {
                auto ctx = std::any_cast<ConnContext>(conn->getContext());
                if (ctx.state != ConnContext::kOpen) continue;

                // 判断空闲超时
                if (now_ms - ctx.last_active_ms > idle_timeout_ms_) {
                    to_close.push_back(conn);
                }
            }
        }
    }

    // 锁外关闭
    for (auto& c : to_close) {
        if (c->connected()) {
            LOG_WARN << "close idle connection";
            c->shutdown();
        }
    }

    // 循环推进分片索引
    idle_scan_shard_idx_ = (idle_scan_shard_idx_ + 1) % idle_scan_shards_;
}

```

3.3 定时触发

构造函数中注册定时任务（每 5 秒触发一次）：

```
CometServer::CometServer(EventLoop* loop, const Config& cfg) {
    // ...
    idle_timeout_ms_ = cfg.comet_idle_timeout_ms; // 默认 60000 (60s)

    if (loop_) {
        loop_>runEvery(5.0, std::bind(&CometServer::CheckIdleConnections, this));
    }
}
```

4. 复杂度分析

假设：

- 总连接数：N
- 分片数：S（默认 10）
- 扫描间隔：T（默认 1s）

单次扫描复杂度

- **理论**： $O(N / S)$ （只扫描约 $1/S$ 的连接）
- **实测**：N = 10,000 连接，S = 10 → 单次扫描约 1,000 条，持锁时间 < 1ms

发现延迟上界

最坏情况：某连接刚好被跳过，需等待下一轮完整循环才能被检测到：

```
延迟上界 = tick_interval × shards
           = 5s × 10
           = 50s
```

为什么可接受？

- 空闲超时阈值通常 60s，50s 的检测延迟相对较小（83.3%）
- 正常断线会立即触发 `onConnection(connection=false)`，无需等待扫描
- 异常断线场景下，50s 延迟不影响业务（Redis TTL 仍会在 60s 后兜底）

锁竞争影响

- **旧方案（已经删除）（全量扫描）**：持锁扫描全部连接，耗时随 N 线性增长
- **新方案（分片扫描）**：持锁仅扫描 N/S 连接，单次持锁时间短且可预测

5. 配置参数

配置项	说明	默认值	建议范围	影响
<code>comet_idle_timeout_ms</code>	空闲超时阈值（毫秒）	60000 (60s)	30000~120000	过小：误杀保活慢的客户端；过大：僵死连接存活时间长
<code>tick_interval</code>	扫描间隔（秒，代码硬编码）	5.0	1~10.0	过小：CPU 开销增加；过大：延迟上界变大

配置项	说明	默认值	建议范围	影响
<code>idle_scan_shards_</code>	分片数（成员变量）	10	5~50	过小：单次扫描量大；过大：发现延迟长

调优建议：

```
// 代码位置：comet/comet_server.h
int idle_scan_shards_{10}; // 可根据连接规模调整

// 如需动态调整 tick_interval，可从配置读取：
// loop_->runEvery(cfg.idle_scan_interval_sec, ...);
```

延迟上界计算示例：

- 配置1: `tick=1s, shards=10` → 延迟上界 10s
- 配置2: `tick=2s, shards=5` → 延迟上界 10s（等效）
- 配置3: `tick=0.5s, shards=20` → 延迟上界 10s（更细粒度，CPU 开销更高）

6. 完整流程示意图

```
时刻 0s: 扫描分片 0 (user_id % 10 == 0)
时刻 1s: 扫描分片 1 (user_id % 10 == 1)
时刻 2s: 扫描分片 2 (user_id % 10 == 2)
...
时刻 9s: 扫描分片 9 (user_id % 10 == 9)
时刻 10s: 扫描分片 0 (循环回到起点)

假设用户 123 (123 % 10 = 3) 在 t=2.5s 时超时：
- t=3s 时扫描分片 3 → 检测到超时 → shutdown()
- 实际延迟：0.5s（最优）

假设用户 123 刚好在 t=3.1s 时超时：
- t=3s 扫描已过，需等到 t=13s 下一轮
- 实际延迟：9.9s（最坏）
```

7. 优缺点总结

✔ 优点：

- 低开销**：活跃连接只更新时间戳，无定时器操作
- 可控抖动**：单次扫描量固定（N/S），不随总连接数抖动
- 锁粒度小**：持锁时间短，对热路径影响小
- 易调优**：分片数/扫描间隔可根据连接规模灵活调整
- 兜底保护**：Redis TTL 在 Comet 异常时仍会清理路由

⚠ 缺点：

- 发现延迟**：最坏情况需等待一轮完整循环（可通过减小 tick 或增加分片缓解）

2. 配置敏感：参数不当可能导致延迟过长或 CPU 浪费

8. 与 Redis TTL 的协同

双层保护机制：



场景对比：

场景	Comet 空闲回收	Redis TTL	结果
客户端正常断开	OnConnection 立即清理	TTL 刷新停止，60s 后过期	路由立即清除 ✓
客户端僵死（有心跳）	last_active_ms 刷新，不触发	TTL 持续刷新	连接保持 ✓
客户端僵死（无心跳）	60s 后检测到超时，主动 shutdown	TTL 刷新停止，60s 后过期	约 70s 内清理 ✓
Comet 崩溃	✗ 无法执行	TTL 刷新停止，60s 后自动清理	依赖 Redis 兜底 ✓

10. 核心要点总结

10.1 消息 ID 设计

msg_id (服务端生成): msgid:1:2-123

↓

严格递增序号 (用于排序)

↓

msg_seq = 123 (从 msg_id 提取)

client_msg_id (客户端生成): c-1701234567-abc

↓

用于 ACK 匹配

10.2 消息流转路径

客户端 → Comet (WebSocket) → Logic (gRPC)

↓

生成 msg_id (Redis INCR)

↓

持久化 (MySQL)

↓

投递 Kafka

↓

Job (消费 Kafka) → Comet (gRPC) → 客户端 (WebSocket)

10.3 路由查询逻辑

Logic 收到消息 → Redis HKEYS user_connections:{target_id} (再提取 comet_id 并去重)

↓

返回 ["comet-1", "comet-2"]

↓

按 comet_id 分组投递 Kafka

↓

Job 消费后调用对应 Comet 的 gRPC

10.4 未读计数公式

$$\text{unread_count} = \text{session.last_msg_seq} - \text{user_read_seq}$$

示例:

- 会话最新消息序号: 123
- 用户已读序号: 118
- 未读数: $123 - 118 = 5$

10.5 Kafka 核心机制

Producer (Logic → Kafka) :

配置关键点：

- `acks=all`：等待所有副本确认（可靠性）
- `enable.idempotence=true`：幂等生产者（防重复）
- `linger.ms=5`：批量发送延迟5ms（性能优化）
- `compression.type=lz4`：LZ4压缩（降低带宽）

分区策略：

- `Key = comet_id`
- 相同 `comet_id` 的消息发到同一分区
- 保证消息顺序性

Consumer (Kafka → Job)：

配置关键点：

- `enable.auto.commit=false`：手动提交 `offset`（防丢失）
- `max.poll.interval.ms=300000`：最大拉取间隔5分钟
- `auto.offset.reset=earliest`：从最早消息开始消费

Pull 模型优势：

- 消费者控制消费速度（背压控制）
- 批量拉取，减少网络开销
- 处理成功后再提交 `offset`（`at least once` 语义）

10.6 gRPC 服务矩阵

服务	作为 gRPC 服务端	作为 gRPC 客户端
Logic	✅ 提供认证、上行消息处理	❌ 无
Comet	✅ 提供下行消息推送接口	✅ 调用 Logic 处理上行消息
Job	❌ 无	✅ 调用 Comet 推送下行消息

调用链路：

上行：客户端 WebSocket → Comet gRPC客户端 → Logic gRPC服务端

下行：Kafka → Job gRPC客户端 → Comet gRPC服务端 → WebSocket

关键设计：

- Stub 缓存：Job 动态管理多个 Comet 的 gRPC Stub
- 线程模型：Logic 在独立线程运行 gRPC，主线程运行 HTTP
- 异步处理：Job 使用线程池处理 Kafka 消息，避免阻塞消费线程

10.7 Comet 连接管理

核心数据结构：

```
std::unordered_map<int64_t, std::set<TcpConnectionPtr>> user_conns_;  
// user_id → 连接集合（支持多设备）
```

多设备设计：

用户A在 PC + 手机登录：

```
user_conns_[1] = { conn_pc, conn_mobile }
```

推送消息时：

- 遍历该用户的所有连接
- 每个设备都收到相同的消息
- 客户端根据 msg_id 去重

离线判断：

- 只有所有连接都断开时才触发 UserOffline
- 避免频繁通知 Logic

生命周期：

1. 握手 → VerifyToken → 获取 user_id
2. 注册 → user_conns_[user_id].insert(conn)
3. 推送 → 遍历 user_conns_[user_id]，批量发送
4. 断开 → user_conns_[user_id].erase(conn)
5. 离线 → 当 set 为空时，触发 UserOffline

线程安全：

- 所有操作加 mutex 保护
- 优化方向：读写锁（读多写少）、连接分片（减少锁粒度）